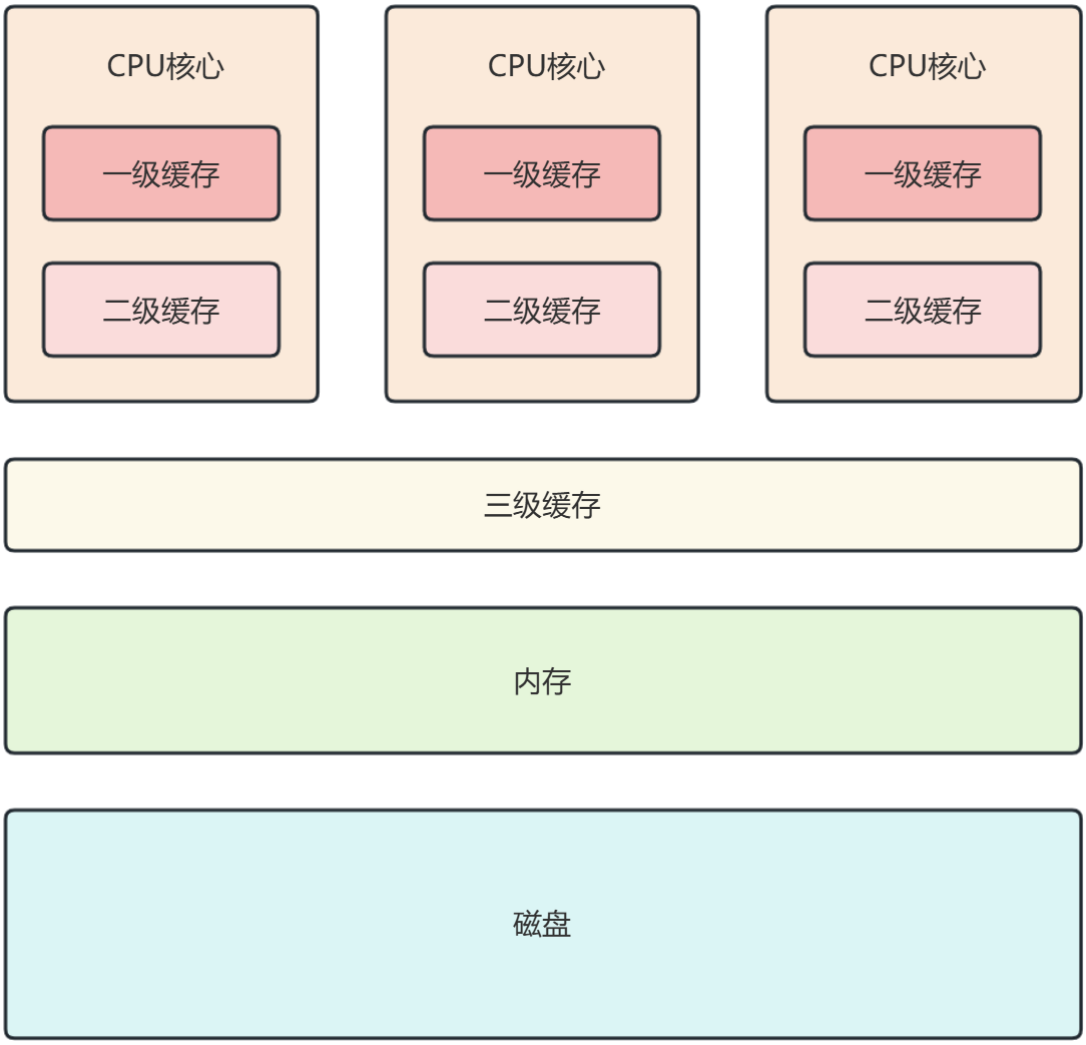
JMM

为什么要有JMM? 什么是JMM?

要聊JMM就得从计算机的发展史说起. 因为磁盘速度太慢了, 所以引入了内存. 运行程序的时候, 先把磁盘中的数据读入内存然后再运行, 这就大大提高了速度. 内存相比于磁盘来说, 其实就是一种缓存. 但是相比cpu来说, 内存还是太慢了, 所以就有了cpu缓存.



有了缓存之后, CPU是怎么工作的呢?

CPU运行程序的时候, 把数据从主存中读到缓存, 然后CPU直接从缓存中读取数据, 运行速度就更快了. 当CPU运算完成后, 再把数据回写到主存中. CPU缓存分3级, 一级和二级是CPU核心私有的, 第三级是共享的.

三大问题

那么问题来了, 当一个CPU核心执行某个线程任务, 它修改了缓存的数据, 但是缓存中数据还没有同步到主存中, 其他CPU核心去执行另一个线程任务, 它从主存中读到的数据不就是脏数据了吗. 一个线程修改了共享数据, 另一个线程无法立刻获取到最新的共享数据, 这就是可见性问题.

那么问题又又来了, 在执行代码的时候, 为了提升执行速度, CPU或编译器可能会对指令进行重排序， 导致执行顺序与代码顺序不一致. 指令重排序可以保证单线程串行执行的语义一致性, 但是无法保证多线程并发执行的语义一致性, 所以在多线程访问共享变量, 指令重排序可能会导致一系列问题, 这就是有序性问题.

那么问题又又又来了, 在多线程并发的环境中, 比如线程A读到库存是1, 想做库存减1的操作, 但是只是读到了, 正准备执行减1操作, cpu时间片耗尽了, 线程B拿到时间片开始执行了, 把库存-1, 此时库存变成0, 然后线程1再执行-1操作, 库存就是-1了, 库存能是负数吗, 肯定不能啊. 所以一个线程去执行一系列不可分割的操作, 但是执行过程却被其他线程插手了, 这就是原子性问题.

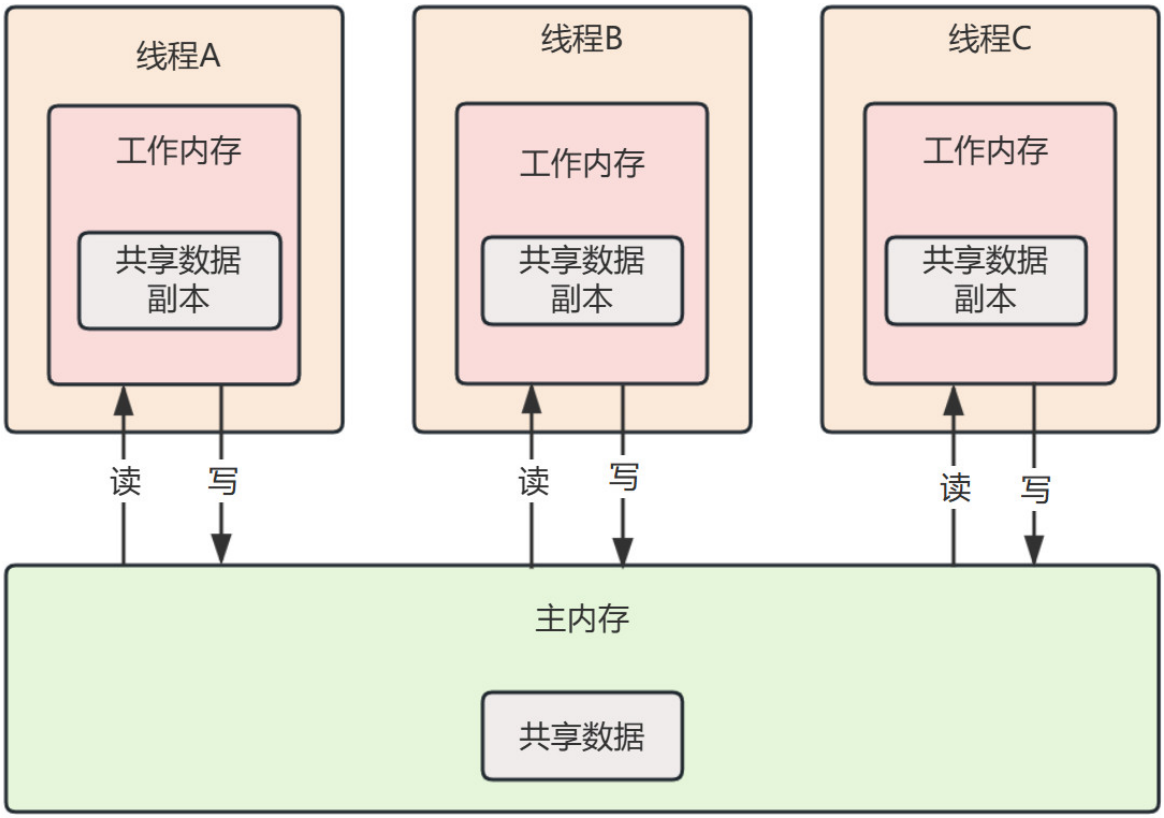
所以整个并发编程核心就是去解决可见性, 有序性, 原子性问题. 操作系统通过内存模型定义一系列规范来解决这些问题. 但是不同的操作系统的内存模型是不同的, 如果直接复用操作系统的内存模型, 就可能会导致同样一套代码换了一个操作系统就无法执行了. Java当时的口号是什么, 一次编译, 处处运行. 为了达到跨平台的目标, 所以Java需要自己提供一套内存模型来屏蔽不同操作系统之间的差异. 这就是java内存模型

那JMM干了啥呢, 怎么解决问题的呢? JMM定义了一套并发编程相关的规范, 抽象了缓存, 线程, 主存之间的关系, 去解决这些问题.

JMM怎么对线程和缓存和主存做的抽象

那他怎么抽象缓存, 线程, 主存的关系呢?

之前聊过, cpu执行命令, 需要把主存中的数据读到缓存中, 在读到寄存器中, 进行计算. 所以JMM将寄存器和缓存等这些私有的东西统一抽象为了线程的工作内存, 这个工作内存就是线程私有的计算区域. 每个线程的工作内存都是独立的，线程操作数据只能在工作内存中进行，然后刷回到主存. 也就是说当每个线程必须把主存中数据读到工作内存再进行计算. 某个线程修改了工作内存, 需要和其他线程共享数据, 必须把自己工作内存的数据刷到主存中, 再让其他线程读取.



抽象出了工作内存之后, JMM就需要规定一套主存与线程工作内存的交互方式来读写数据. 简单来说就是一个共享变量怎么从主内存拷贝到工作内存, 又怎么从工作内存同步到主内存的. 针对这个问题, JMM规定了八种主存与工作内存交互的原子操作.

哪八种呢?

- 第一个, lock, 就是锁定主存变量.
- 第二个, unlock, 解锁主存变量.

- 第三个, read, 从主内存读数据到工作内存中.
- 第四个, load, 将read读到的值放入工作内存变量副本.
- 第五个, use, 将工作内存变量给执行引擎去执行.
- 第六个, assign, 把执行引擎执行的结果赋值给工作内存的变量.
- 第七个, store, 把工作内存的值传输到主存中. 第八个, write, 将store传输过来的值写入主存变量中.

这一套东西大家也不用刻意记, 其实你真正理解了一下子就全会了. 无非就是规定主存怎么和线程工作内存交互, 首先就是能对主存数据加锁, 从主存读到工作内存, 然后把数据赋值给工作内存变量, 然后交给引擎执行, 把执行结果赋值给工作内存变量, 然后把工作内存数据传到主存中, 然后写入主存变量, 最后解锁.

操作	作用
`lock`	锁定主内存变量
`unlock`	解锁主内存变量
`read`	从主内存读取到工作内存传输通道
`load`	将read值放入工作内存变量副本
`use`	工作内存变量传递给执行引擎
`assign`	执行引擎结果赋给工作内存变量
`store`	工作内存值传输到主内存写缓冲区
`write`	将store值写入主内存变量

Java 内存模型还规定上述 8 种基本操作时必须满足如下规则：

- 不允许read和load、store和write操作之一单独出现，即不允许一个变量从主内存读取了但工作内存不接受，或者从工作内存发起回写了但主内存不接受的情况出现。

- 不允许一个线程丢弃它的最近的 assign 操作，即变量在工作内存中改变了之后必须把该变化同步回主内存。
- 不允许一个线程无原因地（没有发生过任何assign操作）把数据从线程的工作内存同步回主内存中。
- 一个新的变量只能在主内存中「诞生」，不允许在工作内存中直接使用一个未被初始化（load或assign）的变量，换句话说，就是对一个变量实施use、store操作之前，必须先执行过了assign和load操作。
- 一个变量在同一个时刻只允许一条线程对其进行lock操作，但lock操作可以被同一条线程重复执行多次，多次执行lock后，只有执行相同次数的unlock操作，变量才会被解锁。
- 如果对一个变量执行lock操作，那将会清空工作内存中此变量的值，在执行引擎使用这个变量前，需要重新执行load或assign操作初始化变量的值。
- 如果一个变量事先没有被lock操作锁定，那就不允许对它执行unlock操作，也不允许去unlock一个被其他线程锁定住的变量。
- 对一个变量执行unlock操作之前，必须先把此变量同步回主内存中（执行store、write操作）。

Java 内存模型还规定上述 8 种基本操作时必须满足一系列的规则. 通过这 8 种内存访问操作以及一系列规则的限定，再加上稍后介绍的对 volatile 的一些特殊规定，就可以确定了 Java 程序中哪些内存访问操作在并发下是安全的.

内存屏障

八种内存操作规定了线程和主存之间交互的方式, 只是定义了可以做什么, 但是没有规定什么时候做. 内存屏障是一种CPU指令, JMM通过插入内存屏障来保证内存操作的顺序和可见性.

JMM定义了四种内存屏障：

- LoadLoad：禁止该屏障后的读操作与该屏障前的读操作重排序
- StoreStore：禁止该屏障后的写操作与该屏障前的写操作重排序
- LoadStore：禁止该屏障后的写操作与该屏障前的读操作重排序
- StoreLoad：禁止该屏障后的读操作与该屏障前的写操作重排序

举个例子, volatile关键字三个特点, 能保证有序性和可见性, 无法保证原子性.

有序性怎么保证的呢? 就是插入了内存屏障, 禁止指令重排序.

可见性就是说一个线程修改的状态对另一 个线程是可见的. 线程每次获取volatile修饰的变量时, 都能获取到最新值. 可见性怎么保证的呢? 其实是在告诉jvm当前变量在工作内存中的值是不确定的，需要从主存中读取, 本质就是禁用了 CPU缓存, 每次去主存取值.

happens-before

八种内存交互操作在底层操作数据, 内存屏障来规定执行顺序. 但是从程序员的角度来说, 你写一段代码还是不知道编译器和处理器怎么重排序的, 怎么优化的. 你不知道实际的执行顺序就不知道线程A做的操作, 线程B是否能看见. 这就很可怕了. 所以JMM通过Happens-Before规则向程序员做一个内存可见性的保证, 如果说操作A happens-before 操作B，那么A的结果对B一定可见.

happens-before 8 条规则, 前五条规则比较重要

- **程序顺序规则**：一个线程内，按照程序代码顺序，书写在前面的操作happens-before于书写在后面的操作
- **锁定规则**：一个unlock操作happens-before于后面对同一个锁的lock操作

- **volatile规则**：对一个volatile变量的写操作happens-before于后面对这个变量的读操作
- **线程启动规则**：Thread对象的start()方法happens-before于此线程的每一个动作。
- **传递性规则**：如果操作A happens-before于操作B，操作B happens-before于操作C，那么操作A happens-before于操作C
- **线程终止规则**：线程中的所有操作都happens-before于对此线程的终止
- **线程中断规则**：对线程interrupt()方法的调用happens-before于被中断线程的代码检测到中断事件的发生，可以通过Thread.interrupted()方法检测到是否有中断发生
- **对象终结规则**：一个对象的初始化完成（构造函数执行结束）happens-before于它的finalize()方法的开始

三大特性如何保证

了解了这些, 再回头看看可见性, 有序性, 原子性三个问题. java在上面一个系列东西的基础上提供一套并发工具, 让开发者能直接使用来解决可见性, 有序性, 原子性三大问题, 从而避免并发安全问题.

最核心的并发工具有哪些呢?

- 第一个, synchronized, 可以保证可见性, 有序性, 原子性.
- 第二, volatile, 可以保证可见性, 有序性.
- 第三, cas, 可以保证原子性. 由cas又衍生出了一系列无锁的并发工具, Atomic原子类, ReentrantLock, CountdownLatch等等.